	CheckGPT	HC3
Training size	720,000*	58,508
Testing size	90,000*	25,049
Avg #words	136.68	145.89
Domains	News, Essay, Research	QA

Table 1: Data statistics, where * denotes the data are randomly split with seed 42, and #words denotes the number of words in one sample.

ing parameters, gradients, training data, and more. Contrastingly, in the **black-box attack** scenario, which aligns more closely with practical applications, the attacker can only access the output results of the victim model. This means only $\mathcal{D}(u, t_{adv})$ or even the binary predictions are provided. Typically, this is further categorized into score-based black-box attacking and decision-based black-box attacking (Wu et al., 2023). In this work, we both consider *white-box attacks* and *decision-based black-box attacks*.

Dynamic Adversarial Attack Another crucial aspect of the detector is its ability to undergo continuous updates using augmentation data from users or other models. As mentioned above, Hu et al. (2023) proposed leveraging a paraphraser as a data generation method to fortify the detector against rewriting attacks during training, resulting in sufficiently robust and transferable detection results. However, this approach remains confined to the model training phase and doesn't explore the robustness and adaptability of the detector in a more dynamic scenario. In this work, we introduce the concept of **dynamic attacks** to iteratively optimize the interplay between attacker and detector across multiple rounds. Detailed processes are shown in Section 4.

3.2 Benchmarking

To validate the performance of adversarial attacks in different scenarios, we selected two datasets: 1) *CheckGPT* (Liu et al., 2023c): Due to the unavailability of the training data division or other details of its detector except for the detector model checkpoint and the full dataset, we consider this scenario as a black box attack, and we partition the entire dataset randomly into 80% training set, 10% validation set, and 10% test set. A surrogate model is trained to act as a proxy for the attacker, and subsequently, we assess the effect of the black box attack on the released original model. 2) *HC3* (Guo et al., 2023): The data partitioning and model parameters are publicly available, making it suitable for a white-box attack. We utilize the public test set as the attack samples and employ the released classifier as the victim model to evaluate the effectiveness of the white-box attack. Further details about both datasets are presented in Table 1.

4 Methodology

In this section, we start by providing an overview and mathematical definitions of our proposed attack methodologies. Following this, we outline the detailed process of establishing a unified framework that bridges the gap between black-box and white-box attack scenarios in Section 4.1 through a surrogate victim model. Furthermore, we delve into the core of our adversarial attack method in Sections 4.4, 4.2, and 4.3 by elucidating the constraints, word importance, and word replacement strategy. Finally, in Section 4.5, we introduce our innovative evaluation paradigm focused on dynamic adversarial attacks.

Algorithm Overview. The HMGC framework we introduce can be conceptualized as an ongoing interaction between the attacker and the detector. When presented with machine-generated text, the attacker iteratively modifies the text in an attempt to fool the detector. This process continues until the detector finally classifies the adversarial text as human-generated. We illustrate the general process in pseudo Algorithm 1. More specifically, the attacker in our HMGC framework comprises four key concepts: the surrogate detection model $\mathcal{D}_\theta$, the word importance ranker $\mathcal{R}$, the encoder-based word swapper $\mathcal{M}$, and a set of constraints $\mathbb{C} = \{c_1, c_2, ..., c_k\}$. The final objective of the ADAT task could be formulated as:

$$\begin{aligned} t_{adv} &= \underset{t_{adv}}{\operatorname{argmin}}\, \mathcal{D}_\theta(t_{adv}), \\ s.t. \quad & t_{adv} \in \{t_{tgt}\} \cup \mathcal{M}(t_{tgt}, R(w_i)), \\ & \sum_{c_i \in \mathbb{C}} \mathbb{1}\left(c_i(t_{adv})\right) = |\mathbb{C}|. \end{aligned} \tag{1}$$

4.1 Surrogate Victim Model

Under the black-box attack setting, obtaining the internal information of the detection model directly is not feasible. To compute the importance of words, it is necessary to train a surrogate model that emulates the behavior of the original detection model and can provide gradients for adversarial attacks

Algorithm 1 HMGC

Input: the original detection model $\mathcal{D}_{ori}$, pre-collected training dataset $\mathbf{T}_C$, a target text t, and an encoder model $\mathcal{M}_{mlm}$, a set of attack constraints $\mathbb{C}$

Parameter: τ threshold for human-written detection, k maximum words can be replaced in one attack

Output: adversarial text t_{adv}

Initialize t_{curr} to represent the current text to be attacked: $t_{curr} \leftarrow t$
procedure 1. Train Surrogate Model
for t_c in $\mathbf{T}_C$ **do**
    $P_D \leftarrow$ predict all t_c using $\mathcal{D}_{ori}$
end for
Train surrogate model $\mathcal{D}_\theta$ on $\mathbf{T}_C$ using P_D as the target label in terms of Eq.2
procedure 2. Pre-attack Assessment
Predict whether the current sample is human-written: $p_h \leftarrow \mathcal{D}_\theta(t_{curr})$
procedure 3. Word Importance Ranking
$W_{curr} = \{w_1, w_2, ..., w_n\} \leftarrow$ Split current sample to words
Calculate word importance I_{w_i} based on gradient and perplexity for each word with Eq.5
$W[:k] \leftarrow$ Sort W_{curr} based on importance
procedure 4. Mask Word Substitution
for w_i in $W[:k]$ **do**
    Replace w_i in t_{curr} with MASK token
    Obtain M candidates $\{p_m\}_{m=1}^{M}$ with Eq.6
    $p_m^* \leftarrow argmax_{m=1}^{M}(\mathcal{D}_\theta(t_{curr} + p_m))$
    $t_{opt} \leftarrow$ Fill in p_m^* for t_{curr}
    if $\mathcal{D}_\theta(t_{opt}) < \mathcal{D}_\theta(t_{curr})$ **then**
        $t_{curr} \leftarrow t_{opt}$
    end if
    Post-checking for t_{curr} with Eq.7 on $\mathbb{C}$
end for
procedure 5. Post-attack Checking
if $\mathcal{D}_\theta(t_{curr}) < \tau$ **or** reach attack limits **then**
    return $t_{adv} = t_{curr}$ ▷ Algorithm terminates
else
    goto procedure 3. ▷ Repeat the process
end if

as a proxy. Following (Guo et al., 2023), we train the surrogate model based on RoBERTa (Liu et al., 2019) with the binary classification task. The training supervision signal is distilled directly from the predictions of the original model. Formally, leveraging a pre-collected training dataset $\mathbf{T}$, we employ the original detection model $\mathcal{D}_{ori}$ to predict each sample, obtaining a set of prediction results, P_D. Subsequently, we initialize the surrogate detection model $\mathcal{D}_\theta$ using the original RoBERTa for training, whereas the training objective is as follows:

$$\mathcal{L}_\theta = -\big(p_i log(\hat{y}_i)\big) + (1 - p_i)\log(1 - \hat{y}_i)), \quad (2)$$

where $p_i \in P_D$, and $\hat{y}_i = \mathcal{D}_\theta(t_i), t_i \in \mathbf{T}$.

4.2 Word Importance Ranking

In our preliminary experiments, we observed that the detector exhibits greater sensitivity to individual words within the text, particularly those that occurred in user requirements. So perturbing important words within the text tends to be more effective in carrying out adversarial attacks. To address this, we have proposed a dual-aspect word importance ranking algorithm that combines model gradients and perplexity derived from large language models. Firstly, it is intuitive that attacks are more effective on tokens with higher gradients on the victim model, whereas higher gradients indicate greater impacts on the final result. Consequently, we consider the gradient norm value corresponding to the *i*-th token as the first aspect of word importance:

$$I_{w_i}^g = \left\| \frac{\partial \mathcal{L}_\theta}{\partial \mathbf{e}_{w_i}} \right\|_1, \quad (3)$$

where $\mathcal{L}_\theta$ is the loss of the objective function; $\mathbf{e}_{w_i}$ is the embedding vector for the *i*-th token in the surrogate model.

Furthermore, existing research (Guo et al., 2023; Liang et al., 2023) on AI-text detection has emphasized the importance of language perplexity as a key indicator for distinguishing between human and machine-generated text. Typically, machine-generated content exhibits lower perplexity. To enhance the effectiveness of our attacks, we introduce additional constraints aimed at increasing the perplexity of the adversarial results, whereas we propose the use of LLM perplexity as a measure for our word importance ranking. More specifically, for each input token, we calculate the perplexity importance as the difference in language perplexity before and after the *i*-th token is removed:

$$I_{w_i}^p = ppl(t_{\backslash w_i}) - ppl(t), \quad (4)$$

where $t_{\backslash w_i}$ represents the text after removing the *i*-th token.

Subsequently, by introducing α as the weighting factor, we obtain the final word importance score:

$$I_{w_i} = (1 - \alpha) I_{w_i}^g + \alpha I_{w_i}^p. \quad (5)$$

4.3 Mask Word Replacements

Here, for an adversarial attack, we sequentially obtain synonymous candidates for each word based on its word importance score in descending order and replace them back into the original text. For instance, for the i-th important token in the text t, we: 1) Replace the token with [MASK]. 2) Utilize the encoder-based model $\mathcal{M}_{mlm}$ to predict the logits for the masked position and perform softmax. 3) Select the top k words with the highest scores as candidates. 4) Replace the candidate words back into the original text one by one, and get the final result following a greedy search strategy. It's worth noting that the source for synonym generation can be any suitable algorithm, such as word embedding spaces (Mrksic et al., 2016) or querying WordNet[3]. However, our early experiments have shown that using an encoder-based model is the most effective way for a random replacement. In summary, we generate candidate synonyms as:

$$p_m^i = \underset{m=1,2,\ldots,k}{\text{softmax}} \left(\mathcal{M}_{mlm}(t_{\setminus w_i} + [MASK]_i) \right). \tag{6}$$

4.4 Attack Quality Constraints

Following the word replacement process, it can be challenging to ensure that the semantics of the original text remain relatively unchanged. For instance, a sentence like "I like that guy" might be transformed into "I hate that guy" after perturbation, resulting in a complete reversal of sentiment. To maintain both syntactic correctness and semantic consistency, we introduce three additional constraints to control the extent of deviation: 1) POS Constraint enforces that the candidate word must align with the part of speech of the word it's replacing, e.g., adjectives cannot be used to substitute nouns. 2) Maximum Perturbed Ratio Constraint limits the proportion of replacement words in the original text within a certain threshold. 3) USE Constraint utilizes the Universal Sentence Encoder (USE) (Cer et al., 2018) as a sentence similarity scorer to measure the distance between the context window of the replacement word and the original text to address the possible semantic shift problem. If the difference is too substantial, the attack is abandoned. In formal terms, for each constraint c_i and the current adversarial text t_{adv}:

$$c_i(t_{adv}) = \begin{cases} true & \text{if } t_{adv} \text{ satisfies } c_i \\ false & \text{else} \end{cases} \tag{7}$$

[3] https://wordnet.princeton.edu/

4.5 Dynamic Detector Finetuning

As mentioned in Section 3.1, in the dynamic attack setting, following each round of attacks that gather a substantial collection of adversarial samples, we proceed to continue training the surrogate model in terms of Eq. 2. This process is designed to strengthen the detector's defense capabilities against one specific form of attack, thereby enabling us to simulate a real-world application scenario where the detector accumulates user queries and continually enhances its capabilities.

5 Experiments

In this section, we first introduce our experimental setup. Next, we compare the performance between HMGC and the baselines in both black-box and white-box attack settings. We then move to the dynamic attack setting, conducting 10 rounds of attack-then-detect iterations to assess the impact of adversarial learning on attack efficacy. Finally, we conduct ablation experiments to analyze the significance of different modules within HMGC.

5.1 Experimental Setup

5.1.1 Evaluation Metrics

Attack performance measures. In line with previous research (Mitchell et al., 2023), we employ the AUC-ROC and the confusion matrix to evaluate the attack performance: 1) **AUC-ROC** is a performance measure that assesses the area under the receiver operating characteristic curve, whereas a higher AUC-ROC score indicates better detection performance. 2) **Confusion matrix** provides a detailed breakdown of the model's performance, with 'positive' denoting human-written content. We report the following three metrics: Positive predictive value (PPV) $\frac{TP}{TP+FP}$, i.e., the proportion of human-written cases among all predicted cases classified as human-written. True negative rate (TNR) $\frac{TN}{TN+FP}$, i.e., the accuracy in classifying machine-generated text. We also denote the decrease of TNR as ΔAcc, which quantifies the reduction in the accuracy of machine-generated sample detection after the attack. It is calculated as $\frac{\text{TNR before Attack} - \text{TNR after Attack}}{\text{TNR before Attack}}$.

Text quality measures. Here, we use the following metrics to evaluate the text quality after the adversarial attack. **Flesch reading ease**: Higher Flesch scores indicate that the material is easier to read. To assess the impact of text's readability, we use the difference ratio of the Flesch score, denoted as